

EventHub

College Event Ticketing Platform - Project Guide & Phase Plan

Practice project for the Java Full Stack journey

Tech: React + Spring Boot + MySQL

Owner: Madhavan | Version 1.0 - September 2026

1. Project Overview

EventHub is a full stack web application where college students discover and book tickets for college events, clubs (organizers) create and manage events, and admins monitor everything. Think of it as "**BookMyShow for your college**".

Goal of the project: learn and practise the complete Java full stack (React + Spring Boot + MySQL) by building one real, working product before starting the main e-commerce project.

The problem it solves

- Events are announced on notice boards and WhatsApp groups, so students miss them.
- Registration happens through Google Forms, with payment checked manually from UPI screenshots.
- Organizers track participants in Excel and do not know how many seats are left.
- Nobody can easily see how much money each event collected.

EventHub puts all of this into one website.

Who uses it

User	What they can do
Student	Register, log in, browse / search / filter events, add tickets to the cart, pay online, see My Bookings, cancel a booking
Organizer (club / department)	Create events (title, date, venue, price, seats), edit or delete their own events, see who booked
Admin (college staff)	See all events and bookings, revenue reports and contact messages; manage users

Example story

1. The Coding Club (Organizer) creates "Hackathon 2026" - 15 Oct, Main Auditorium, Rs.100, 200 seats.
2. Ravi (Student) filters by "Tech" and sees Hackathon 2026 with 200 seats left.
3. Ravi clicks Book; the ticket goes into his cart; he pays Rs.100 on the Stripe page.
4. The payment succeeds, the booking becomes PAID and the seats drop to 199.
5. Ravi sees the ticket in My Bookings.
6. The Admin dashboard shows "Hackathon 2026 - 1 booking - Rs.100 collected".

2. Tech Stack - What We Are Going to Use

Frontend - what the user sees

Tool	What it does	Why we use it
React	JavaScript library for building UIs	The most in-demand frontend skill
Vite	Creates and runs the React project fast	Faster and simpler than older tools
React Router	Moves between pages without reloading	Makes the site feel like an app (SPA)
Tailwind CSS	Styling with ready-made classes	Beautiful, responsive design quickly
Redux Toolkit	Stores shared data (cart, logged-in user)	Every page can reach the same data
Axios	Sends requests from React to the backend	Easier than plain fetch; attaches the JWT

Backend - the brain and the rules

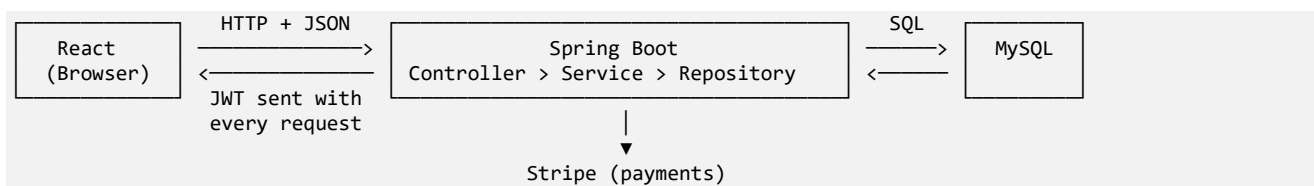
Tool	What it does	Why we use it
Java 21	The programming language	Strong, stable, used by big companies
Spring Boot 3	Framework for building the backend quickly	Industry standard for Java backends
Spring Web	Creates REST APIs	Lets React talk to Java
Spring Data JPA	Saves and reads Java objects in the database	Less SQL to write by hand
Spring Security + JWT	Login and role-based protection	Real-world security
Validation	Checks the input (email format etc.)	Stops bad data getting in
springdoc (Swagger)	Automatic API documentation page	Easy testing and explaining
Spring Cache	Remembers frequent results	A faster app
Actuator	Health and monitoring endpoints	Production-style monitoring
Maven	Manages libraries and builds the project	Handles dependencies automatically

Database & other tools

Tool	Use
MySQL	Stores users, events and bookings
H2	Temporary in-memory database for early testing
Docker	Runs MySQL without a complicated install
Stripe (test mode)	Online payments - no real money
Postman	Tests APIs before connecting React
Git + GitHub	Version control and portfolio

Tool	Use
IntelliJ IDEA / VS Code	Editors for Java / React

3. Architecture - How It Works



- **Controller** - receives the request ("give me all events")
- **Service** - applies the business rules ("are seats available?")
- **Repository** - talks to the database

4. Features

Students

- Register and log in
- Browse events; search and filter by category and date
- Event details with seats left
- Add tickets to the cart and pay online (Stripe)
- My Bookings with cancel option
- Dark mode

Organizers

- Create, edit and delete their own events
- See the bookings for their events

Admins

- Dashboard - total bookings, revenue per event, contact messages
- Manage all events and users
- Access to Swagger and Actuator (secured)

5. Database Design

```

users —1:1— profiles      (phone, college, department)
users —M:N— roles         (STUDENT, ORGANIZER, ADMIN)
users —1:N— bookings
events —1:N— bookings
events —M:N— tags         (tech, cultural, sports)
users(organizer) —1:N— events
contact_messages
  
```

Table	Main columns
users	id, name, email, password (BCrypt hash), created_at
profiles	id, user_id, phone, college, department
roles / user_roles	id, name user_id, role_id
events	id, title, description, venue, date_time, price, capacity, seats_left, category, organizer_id
tags / event_tags	id, name event_id, tag_id
bookings	id, user_id, event_id, quantity, amount, status (PENDING / PAID / CANCELLED), stripe_session_id, created_at
contact_messages	id, name, email, message, created_at

This single design covers **every JPA relationship** - a favourite interview topic.

6. REST API List

Method	Endpoint	Who	Purpose
POST	/api/auth/register	Public	Create an account
POST	/api/auth/login	Public	Log in and get a JWT
GET	/api/events?category=&q=	Public	List / search events
GET	/api/events/{id}	Public	Event details
POST	/api/events	Organizer	Create an event
PUT	/api/events/{id}	Organizer (owner)	Update an event
DELETE	/api/events/{id}	Organizer / Admin	Delete an event
POST	/api/bookings/checkout	Student	Create booking + Stripe session
GET	/api/bookings/me	Student	My bookings
PATCH	/api/bookings/{id}/cancel	Student	Cancel a booking
GET	/api/admin/bookings	Admin	All bookings
GET	/api/admin/stats	Admin	Revenue and counts
GET	/api/admin/messages	Admin	Contact messages
POST	/api/contact	Public	Send a message

7. Folder Structure

This project folder (D:\Java_FullStack_Journey\02_EventHub_Practice_Project) already contains empty **backend** and **frontend** folders ready for the code.

```
02_EventHub_Practice_Project/
├── backend/                               (Spring Boot)
│   └── src/main/java/com/eventhub/
│       ├── controller/                   APIs
│       ├── service/                     business logic
│       ├── repository/                  database access
│       ├── entity/                     tables as Java classes
│       ├── dto/                        data sent to / from React
│       ├── security/                   JWT and login
│       ├── exception/                 error handling
│       └── config/                     settings
├── frontend/                             (React + Vite)
│   └── src/
│       ├── components/                 Navbar, EventCard...
│       ├── pages/                     Home, Events, Login...
│       ├── store/                     Redux (cart, auth)
│       ├── api/                       Axios calls
│       └── routes/                     page routing
└── docker-compose.yml                  (MySQL)
```

8. Phase-wise Plan

Total: about **10-12 weeks** at 1-2 hours per day. Finish each phase's "Done when" check before moving on.

Phase 0 - Foundations (2-3 weeks)

Goal: Get comfortable with the basics (see the Learning Notes document).

- Java basics, OOP, collections, exceptions - small console programs
- Build a console "Event Manager" (add / list / search events with ArrayList and classes)
- JavaScript practice - arrays, objects, async/await, fetch
- SQL practice - SELECT, JOIN, GROUP BY
- Git basics - init, add, commit, push

Done when: The console Event Manager works and is pushed to GitHub.

Phase 1 - Setup & Design (3-4 days)

Goal: Everything installed and the project planned.

- Install JDK 21, IntelliJ, VS Code, Node.js, Docker Desktop, Postman, Git
- Draw the database tables on paper
- Sketch the pages: Home, Events, Event Details, Login, Cart, My Bookings, Admin
- Create the GitHub repository
- Generate the Spring Boot project (start.spring.io) into backend/ and the React app (Vite) into frontend/

Done when: Both "Hello World" apps run.

Phase 2 - Frontend UI (1 week)

Goal: Static React pages with dummy data.

- Components: Navbar, EventCard, EventList, Footer
- Style with Tailwind CSS; dark mode toggle
- React Router pages: Home, Events, Details, Login, Register, 404

Done when: You can click through every page with fake data.

Phase 3 - Backend REST API (1-1.5 weeks)

Goal: Event CRUD in Spring Boot.

- Event entity, EventRepository, EventService, EventController
- Start with H2, then move to MySQL in Docker
- DTOs, validation (@Valid), global exception handler
- Swagger docs; test everything in Postman

Done when: GET / POST / PUT / DELETE on /api/events work in Postman.

Phase 4 - Connect Frontend & Backend (4-5 days)

Goal: React shows real data from the database.

- Axios API calls; enable CORS in Spring
- Loading and error states
- Search and filter by category

Done when: Events added in MySQL appear in the React UI.

Phase 5 - Authentication & Roles (1.5 weeks)

Goal: Secure login and role-based access.

- User, Role and Profile entities (One-to-One, Many-to-Many)
- Register and login APIs with BCrypt password hashing
- Issue JWT tokens; protect routes in Spring and React
- Roles: STUDENT, ORGANIZER, ADMIN

Done when: A student cannot open admin pages (403); an admin can.

Phase 6 - Booking & Cart (1 week)

Goal: Students can book tickets.

- Cart with Redux Toolkit
- Booking entity and API
- Seat-count update with @Transactional
- My Bookings page with cancel option

Done when: Booking and cancelling correctly change the seats left.

Phase 7 - Payments (1 week)

Goal: Paid events go through Stripe.

- Stripe Checkout in test mode (success and cancel pages)
- Update the booking status to PAID after payment
- Free events are booked directly

Done when: The test card 4242 4242 4242 4242 completes a booking.

Phase 8 - Admin & Advanced Features (1 week)

Goal: Real-world polish.

- Admin dashboard: all bookings, revenue per event (JPQL query), messages
- Caching with @Cacheable / @CacheEvict on the event list
- dev and prod profiles; secret keys in environment variables
- CSRF protection; secure Swagger and Actuator

Done when: The admin sees live stats and the event list loads from the cache.

Phase 9 - Testing, Deployment & Portfolio (1 week)

Goal: Make it resume-ready.

- Unit tests with JUnit and Mockito for the services
- Deploy: backend on Render / Railway, database on a cloud MySQL, frontend on Vercel / Netlify
- README with screenshots, tech stack and live link
- Demo video / LinkedIn post

Done when: A recruiter can open the live link and try the app.

9. Progress Tracker

Fill in the Status column as you go (Not started / In progress / Done).

Phase	Work	Planned time	Start date	Status
0	Java, JS and SQL foundations	2-3 weeks		
1	Setup and design	3-4 days		
2	React UI	1 week		
3	Spring Boot REST API	1-1.5 weeks		
4	Connect frontend and backend	4-5 days		
5	Login, JWT and roles	1.5 weeks		
6	Cart and bookings	1 week		
7	Stripe payment	1 week		
8	Admin, caching, security polish	1 week		
9	Testing, deployment, README	1 week		

10. Benefits of This Project

Benefit	Explanation
Complete full stack practice	Frontend, backend, database, security and payments in one project
Easy to understand	You already know how college events work
Interview ready	Covers JWT, JPA relationships, REST, hooks, Redux, @Transactional
Resume project	"Built a college event ticketing platform using React & Spring Boot"
Final-year / mini project	Can be submitted in college
Real use	College clubs could actually use it
Prepares for the main project	About 70-80% of the e-commerce skills are practised here

11. Skills You Will Gain

- **Java:** OOP, collections, exceptions, streams
- **Spring:** dependency injection, REST, JPA, Security, JWT, caching, profiles
- **React:** components, hooks, routing, Redux, Tailwind
- **Database:** MySQL, relationships, JPQL, transactions
- **Tools:** Git, Docker, Postman, Swagger
- **Soft skills:** planning, debugging, reading documentation

12. Before You Start

Category	Needed
Knowledge	Basic Java (OOP), HTML / CSS / JavaScript basics, a little SQL
Software	JDK 21, IntelliJ IDEA Community, VS Code, Node.js (LTS), Docker Desktop, Postman, Git
Accounts (free)	GitHub; Stripe (test mode, only needed in Phase 7)
Laptop	8 GB RAM or more recommended (for Docker)

13. Final Checks - "I really learned it"

- I can explain the full flow: React ► Axios ► Controller ► Service ► Repository ► MySQL
- Test card 4242 4242 4242 4242 completes a booking and the status changes to PAID
- A STUDENT token gets 403 on /api/admin/*, an ADMIN token gets 200
- The second call to the event list is served from the cache
- The project is on GitHub with a README and a live demo link

Teacher's tips

1. Do not skip Phase 0.
2. Type the code yourself - never copy-paste.
3. Commit to GitHub every day.
4. After each phase, explain it out loud as if in an interview.